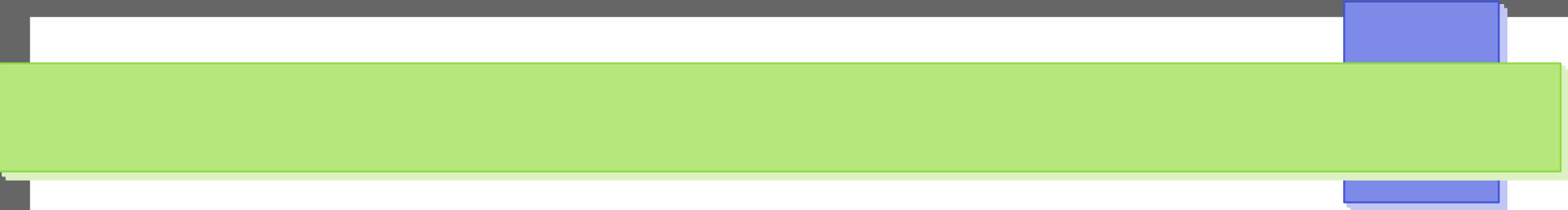


Physical Database Design II

Indexing Structures for Files



Adapted from

Outline

- Introduction
- Types of Single-Level Ordered Indexes
- Multilevel Indexes
- Dynamic Multilevel Indexes Using B-Trees and B+-Trees
- Indexes on Multiple Keys
- Other Types of Indexes
- Some General Issues Concerning Indexing

Introduction

- Indexes used to speed up record retrieval in response to certain search conditions
- Index structures provide secondary access paths
- Any field can be used to create an index
 - Multiple indexes can be constructed
- Most indexes based on ordered files

Outline

- Introduction
- Types of Single-Level Ordered Indexes
- Multilevel Indexes
- Dynamic Multilevel Indexes Using B-Trees and B+-Trees
- Indexes on Multiple Keys
- Other Types of Indexes
- Some General Issues Concerning Indexing

Single-Level Ordered Indexes

- Ordered index similar to index in a textbook
- Textbook
 - Search the book index for a certain term in the textbook to find a list of addresses, i.e., page numbers
- For a file with a given record structure consisting of several fields (or attributes)
 - Index access structure is usually defined on a single field (the indexing field)
 - Index stores each value of the index field with list of pointers to all disk blocks that contain records with that field value
- Values in index are ordered

Single-Level Ordered Indexes

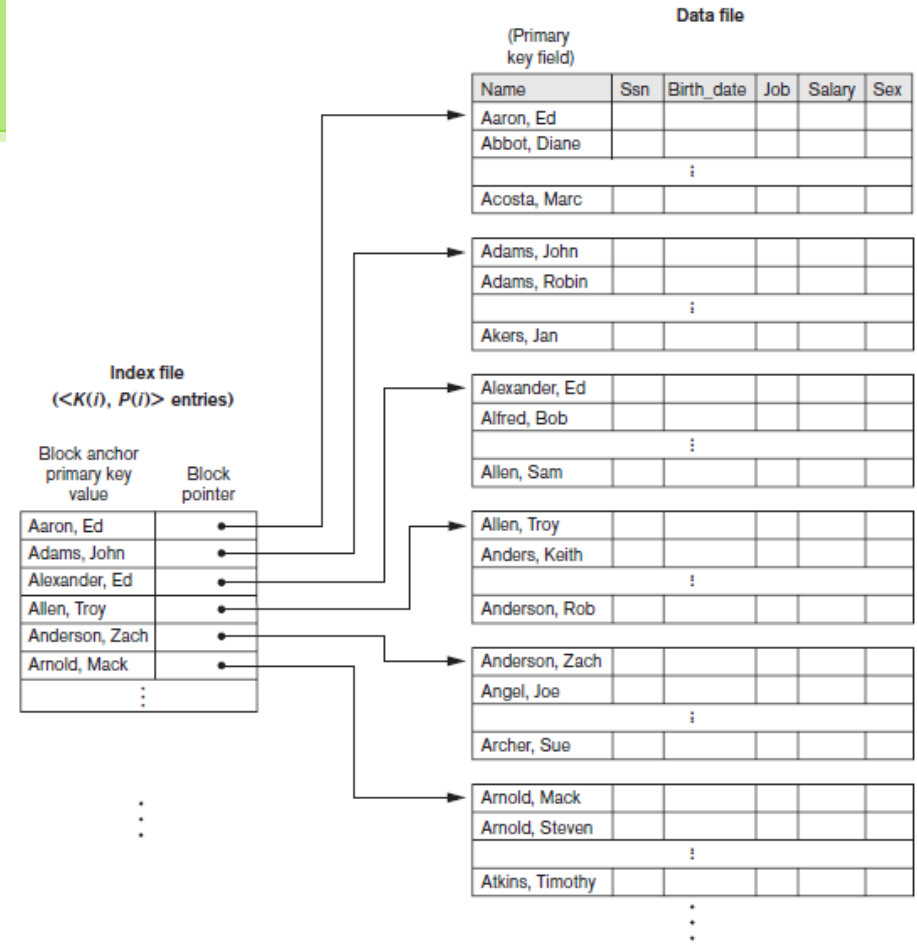
- Several types of ordered indexes
- Primary index
 - Specified on the ordering key field of ordered file of records
- Clustering index
 - Used if numerous records can have the same value for the ordering field
- Secondary index
 - Can be specified on any non-ordering field
 - Data file can have several secondary indexes

Primary Indexes

- Ordered file with two fields
 - Primary key, $K(i)$
 - Pointer to a disk block, $P(i)$
- One index entry in the index file for each block in the data file
- Indexes may be dense or sparse
 - Dense index has an index entry for every search key value in the data file
 - Sparse index has entries for only some search values

Primary Indexes

- Example: Primary index on the ordered file on the right
- $\langle K(1) = (\text{Aaron, Ed}), P(1) = \text{address of block 1} \rangle$
 $\langle K(2) = (\text{Adams, John}), P(2) = \text{address of block 2} \rangle$
 $\langle K(3) = (\text{Alexander, Ed}), P(3) = \text{address of block 3} \rangle$
- The total number of entries in the index is the same as the number of disk blocks in the ordered data file
 - First record in each block of the data file is called the anchor record



Primary Indexes

- The index file for a primary index occupies a much smaller space than does the data file
 - Fewer index entries than there are records in the data file
 - Each index entry is typically smaller in size than a data record because it has only two fields
- Consequently, more index entries than data records can fit in one block
- Hence, a binary search on the index file requires fewer block accesses than a binary search on the data file
 - For an ordered data file with b blocks, $\log_2 b$ block accesses
 - For primary index file with only b_i blocks
 - To locate a record with a search key value requires a binary search of that index, and then access to the block containing that record: a total of $\log_2 b_i + 1$ accesses

Primary Indexes

- Major problem: insertion and deletion of records
 - Records are moved around to make space for the new record
 - Index entries may also need to be changed
- Solutions
 - Use unordered overflow file
 - Use linked list of overflow records

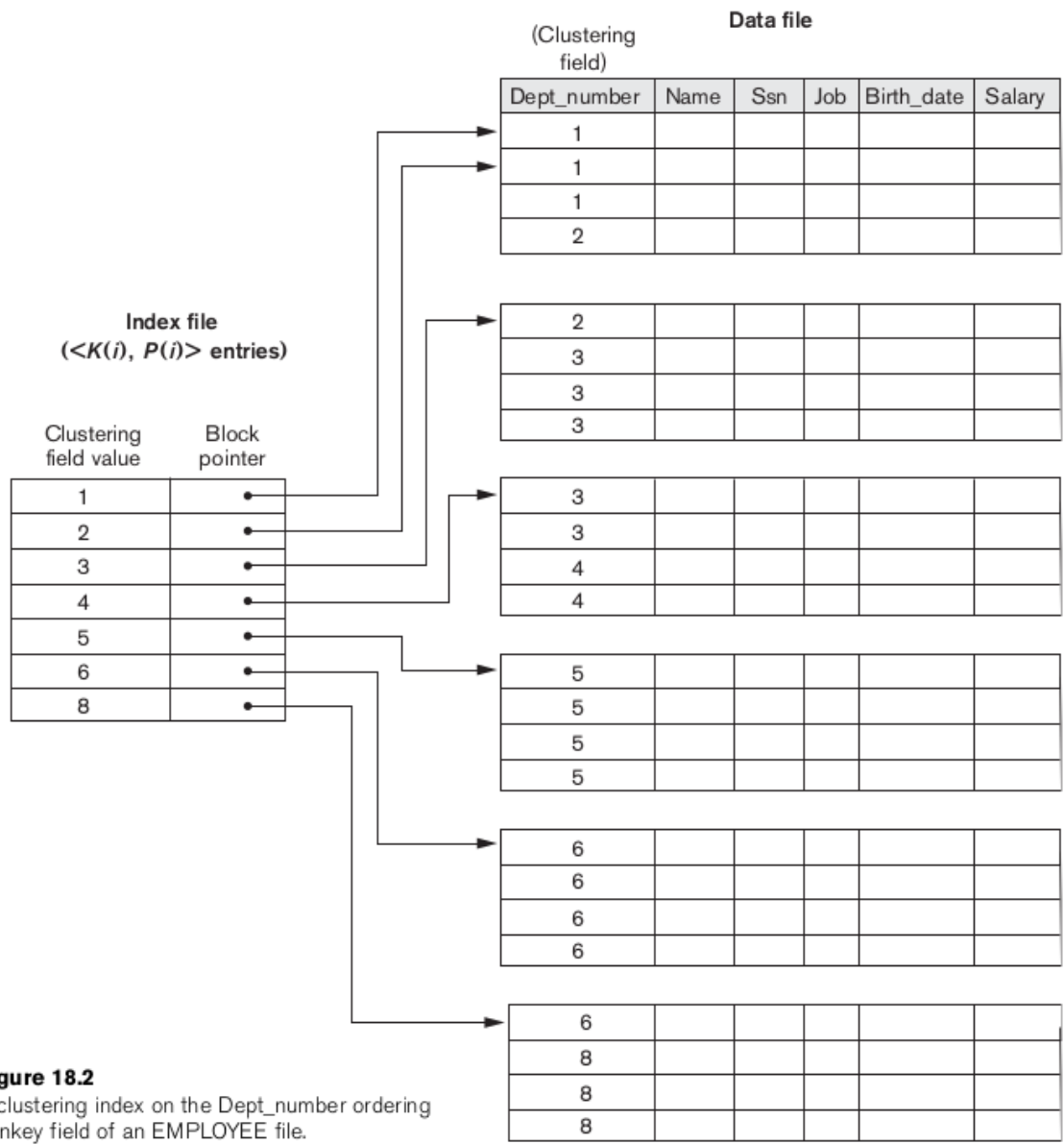
Clustering Indexes

- If file records are physically ordered on a non-key field - which does not have a distinct value for each record
 - That field is called the clustering field
 - The data file is called a clustered file
- Clustering index can be created to speed up retrieval of all the records that have the same value for the clustering field
 - Different from primary index, which requires that the ordering field of the data file have a distinct value for each record

Clustering Indexes

- Clustering index
 - An ordered file with two fields
 - First field is of the same type as the clustering field of the data file
 - Second field is a disk block pointer
 - One entry in the clustering index for each distinct value of the clustering field, and it contains the value and a pointer to the first block in the data file that has a record with that value for its clustering field

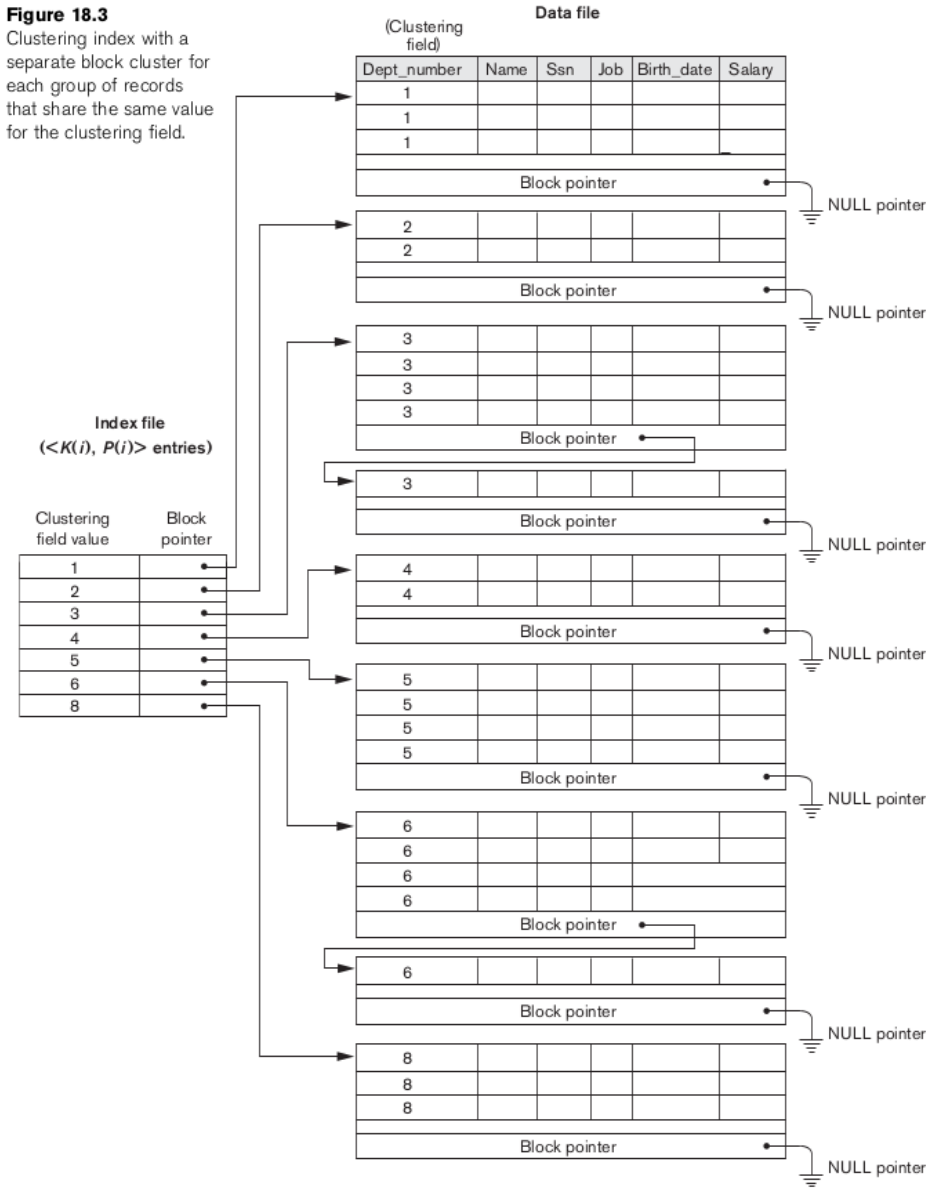
Clustering Indexes



Clustering Indexes

Figure 18.3

Clustering index with a separate block cluster for each group of records that share the same value for the clustering field.



Secondary Indexes

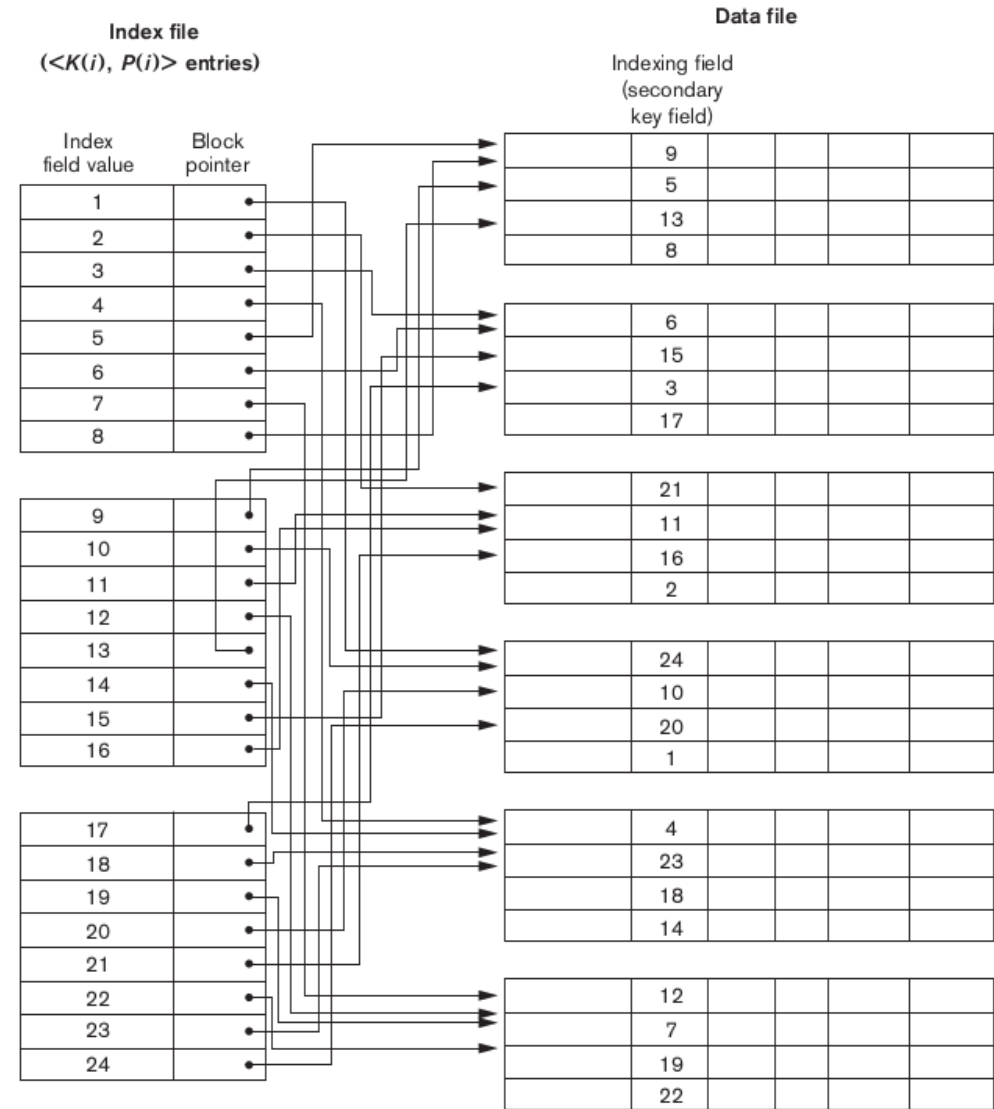
- Provide secondary means of accessing a data file for which some primary access exists
- Secondary index
 - Ordered file with two fields
 - Indexing field, $K(i)$
 - Same data type as some non-ordering field of the data file that is an indexing field
 - Block pointer or record pointer, $P(i)$
 - Usually need more storage space and longer search time than primary index
 - But improved search time for arbitrary record

Figure 18.4

A dense secondary index (with block pointers) on a nonordering key field of a file.

Secondary Indexes

- Index entries are ordered by value of $K(i)$, so we can perform a binary search
- Because the records of the data file are not physically ordered by values of the secondary key field, we cannot use block anchors
 - Reason why an index entry is created for each record in the data file, rather than for each block, as in the case of a primary index
- Note
 - In this example, pointers $P(i)$ are block pointers, not record pointers
 - Point to the block that contain the data



Outline

- Introduction
- Types of Single-Level Ordered Indexes
- **Multilevel Indexes**
- Dynamic Multilevel Indexes Using B-Trees and B+-Trees
- Indexes on Multiple Keys
- Other Types of Indexes
- Some General Issues Concerning Indexing

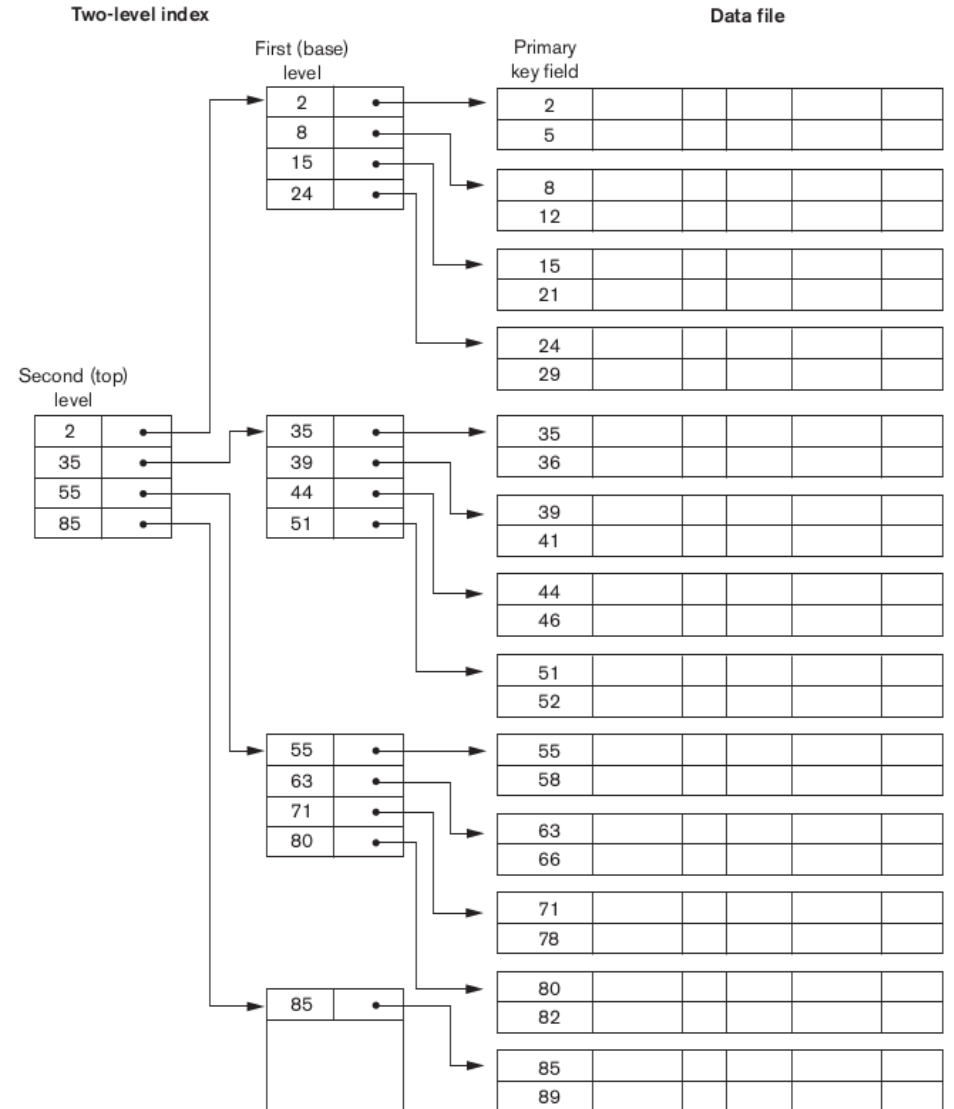
Multilevel Indexes

- The indexing schemes we have described thus far
 - Involve an ordered index file
 - A binary search is applied to the index to locate pointers to a disk block or to a record (or records) in the file having a specific index field value
- A binary search requires approximately $(\log_2 b_i)$ block accesses for an index with b_i blocks
 - After binary search of the index, we search just the block containing our data – perhaps using linear search

Multilevel Indexes

- Idea behind a multilevel index
 - Reduce the part of the index that we continue to search by bfr_i , the blocking factor for the index
 - This way, the search space is reduced much faster
 - bfr_i is called the fan-out of the multilevel index (we will refer to it by the symbol fo)
- Note
 - Binary search divides the record search space into two halves at each step of the search
 - Multilevel search divides the search space n -ways (where n = the fan-out) at each search step using the multilevel index
 - Searching a multilevel index requires approximately $(\log_{fo} b_i)$ block accesses
 - This is a substantially smaller number than for a binary search if the fan-out is larger than 2
 - In most cases, the fan-out is much larger than 2

Multilevel Indexes



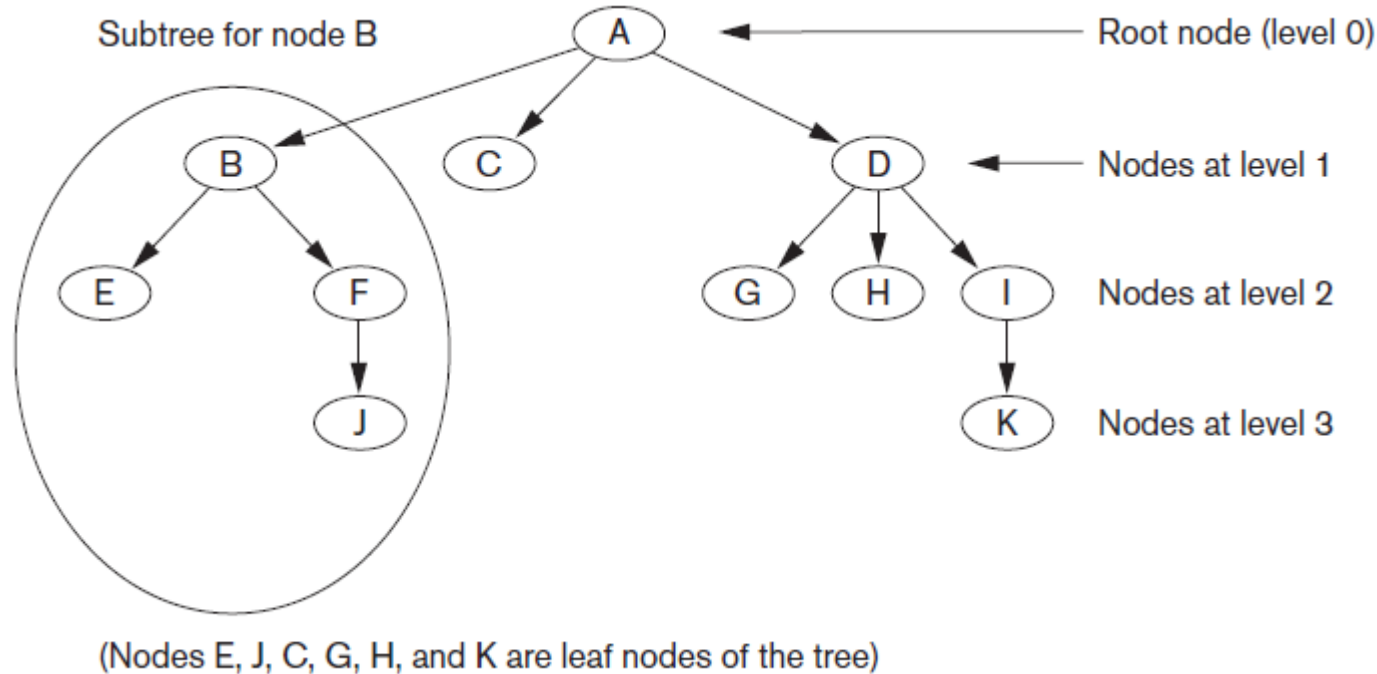
Outline

- Introduction
- Types of Single-Level Ordered Indexes
- Multilevel Indexes
- **Dynamic Multilevel Indexes Using B-Trees and B+-Trees**
- Indexes on Multiple Keys
- Other Types of Indexes
- Some General Issues Concerning Indexing

Dynamic Multilevel Indexes Using B-Trees and B+-Trees

- B-trees and B⁺-trees are special cases of the well-known **search tree** data structure
- What is a tree?
 - Tree is formed of nodes
 - Each node (except root) has one parent and zero or more child nodes
 - Leaf node has no child nodes
 - Unbalanced if leaf nodes occur at different levels
 - Nonleaf node called internal node
 - Subtree of node consists of node and all descendant nodes

Dynamic Multilevel Indexes Using B-Trees and B+-Trees



Unbalanced tree example

Search Tree and B-Trees

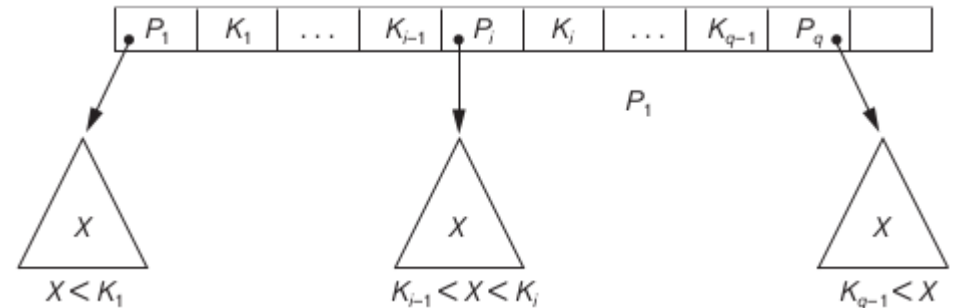
- Search tree
 - Special type of tree that is used to guide the search for a record, given the value of one of the record's fields
 - Multilevel indexes we saw earlier is a variation of a search tree
 - Each node in the multilevel index can have as many as fo pointers and fo key values (fo is the index fan-out)
 - The index field values in each node guide us to the next node, until we reach the data file block that contains the required records
 - By following a pointer, we restrict our search at each level to a subtree of the search tree and ignore all nodes not in this subtree

Search Tree and B-Trees

- A search tree is slightly different from a multi-level list
 - A search tree of order p is a tree such that
 - Each node contains at most $p-1$ search values and p pointers in the order $\langle P_1, K_1, P_2, K_2, \dots, P_{q-1}, K_{q-1}, P_q \rangle$, where $q \leq p$
 - Each P_i is a pointer to a child node (or a NULL pointer)
 - Each K_i is a search value from some ordered set of values
 - (see more details in textbook)

Search Tree and B-Trees

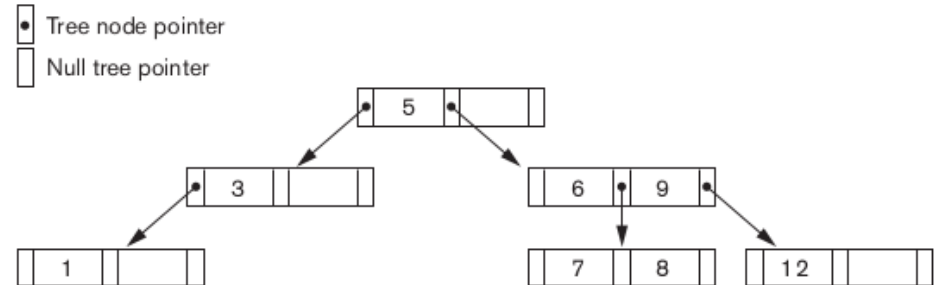
- A node in a search tree is shown on the right
- Search tree constraints given below



1. Within each node, $K_1 < K_2 < \dots < K_{q-1}$.
2. For all values X in the subtree pointed at by P_i , we have $K_{i-1} < X < K_i$ for $1 < i < q$; $X < K_i$ for $i = 1$; and $K_{i-1} < X$ for $i = q$ (see Figure 18.8).

Search Trees and B-Trees

- A search tree of order 3 (max of 2 data nodes)
- Search tree can be used as a mechanism to search for records stored in a disk file.
 - The values in the tree can be the values of the search field
 - Same as the index field if a multilevel index guides the search
 - Each key value in the tree is associated with a pointer to the record in the data file having that value
 - When a new record is inserted in the file, we must update the search tree by inserting an entry in the tree containing the search field value of the new record and a pointer to the new record



Search Tree and B-Trees

- Algorithms are necessary for inserting and deleting search values into and from the tree
- These algorithms do not guarantee that a search tree is balanced (by having leaf nodes at the same level)
- In a balanced tree
 - Nodes are evenly distributed, so that the depth of the tree is minimized for the given set of keys and the tree does not get skewed with some nodes being at very deep levels
 - Search speed is uniform (average time to find any random key is roughly the same)
- Another goal
 - The tree should not need too much restructuring as records are inserted into and deleted from the main file
 - Achievable by
 - Making sure the nodes to be as full as possible
 - Avoiding empty nodes when many deletions are made

Search Tree and B-Trees

- B-tree
 - Like search tree, provides multi-level access structure
 - Has additional constraints that ensure that
 - Tree is always balanced
 - Space wasted by deletion never becomes excessive
 - Each node is at least half-full
 - Each node in a B-tree of order p can have at most $p-1$ search values

Search Tree and B-Trees

- Each internal node in the B-tree (Figure 18.10(a)) is of the form

$$\langle P_1, \langle K_1, Pr_1 \rangle, P_2, \langle K_2, Pr_2 \rangle, \dots, \langle K_{q-1}, Pr_{q-1} \rangle, P_q \rangle$$

where $q \leq p$. Each P_i is a **tree pointer**—a pointer to another node in the B-tree. Each Pr_i is a **data pointer**⁸—a pointer to the record whose search key field value is equal to K_i (or to the data file block containing that record).

- Within each node, $K_1 < K_2 < \dots < K_{q-1}$.
- For all search key field values X in the subtree pointed at by P_i (the i th subtree, see Figure 18.10(a)), we have:
 $K_{i-1} < X < K_i$ for $1 < i < q$; $X < K_i$ for $i = 1$; and $K_{q-1} < X$ for $i = q$.
- Each node has at most p tree pointers.

- Each node, except the root and leaf nodes, has at least $\lceil (p/2) \rceil$ tree pointers. The root node has at least two tree pointers unless it is the only node in the tree.
- A node with q tree pointers, $q \leq p$, has $q - 1$ search key field values (and hence has $q - 1$ data pointers).
- All leaf nodes are at the same level. Leaf nodes have the same structure as internal nodes except that all of their *tree pointers* P_i are NULL.

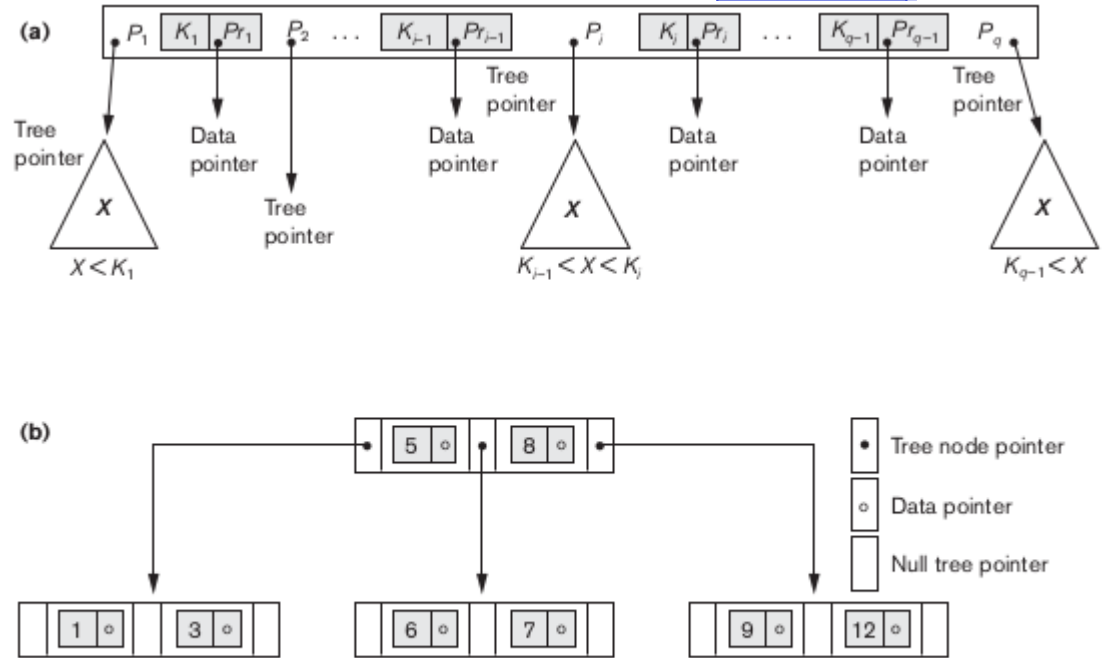


Figure 18.10

B-tree structures. (a) A node in a B-tree with $q - 1$ search values. (b) A B-tree of order $p = 3$. The values were inserted in the order 8, 5, 1, 7, 3, 12, 9, 6.

Search Tree and B-Trees

- Constructing a B-tree
 - Starts with a single root node (which is also a leaf node) at level 0 (zero).
 - Once the root node is full with $p - 1$ search key values and we attempt to insert another entry in the tree
 - Root node splits into two nodes at level 1
 - Only the middle value is kept in the root node, and the rest of the values are split evenly

Search Tree and B-Trees

- Constructing a B-tree
 - When a non-root node is full and a new entry is inserted into it
 - That node is split into two nodes at the same level, and the middle entry is moved to the parent node along with two pointers to the new split nodes
 - If the parent node is full, it is also split
 - Splitting can propagate all the way to the root node, creating a new level if the root is split

B⁺ Tree

- Most implementations of a dynamic multilevel index use a variation of the B-tree data structure called a B⁺-tree
 - In a B-tree, every value of the search field appears once at some level in the tree, along with a data pointer
 - In a B⁺-tree, data pointers are stored only at the leaf nodes of the tree
 - See more details in textbook

Outline

- Introduction
- Types of Single-Level Ordered Indexes
- Multilevel Indexes
- Dynamic Multilevel Indexes Using B-Trees and B+-Trees
- **Indexes on Multiple Keys**
- Other Types of Indexes
- Some General Issues Concerning Indexing

Indexes on Multiple Keys

- Multiple attributes are involved in many retrieval and update requests
- Composite keys
 - Access structure using key value that combines attributes
- Partitioned hashing
- Suitable for equality comparisons

Indexes on Multiple Keys

- Grid Files

- Consider Employee file
- We can organize the EMPLOYEE file as a grid file
- To access a file on two keys, say Dno and Age, we can construct a grid array with one linear scale (or dimension) for each of the search attributes

Dno

0	1, 2
1	3, 4
2	5
3	6, 7
4	8
5	9, 10

**Linear scale
for Dno**

EMPLOYEE file

5						
4						
3						
2						
1						
0						
	0	1	2	3	4	5

Bucket pool

Bucket pool

Linear Scale for Age

0	1	2	3	4	5
< 20	21-25	26-30	31-40	41-50	> 50

Indexes on Multiple Keys

- Each scale is made in a way as to achieve a uniform distribution of that attribute
- In our example
 - Linear scale for DNo has DNo = 1, 2 combined as one value 0 on the scale
 - DNo = 5 corresponds to the value 2 on that scale, etc.
 - Linear scale for Age: 0 to 5
 - Ages are similarly grouped so as to distribute the employees uniformly by age
 - 36 cells in grid array for this file
 - Each cell points to some bucket address where the records corresponding to that cell are stored
 - Hence query: List the employees in department number 4 whose age is 59
 - Maps into the cell (1, 5) of the grid array
 - The records for this combination will be found in the corresponding bucket

Indexes on Multiple Keys

- Grid file contd.
 - This method is particularly useful for range queries that would map into a set of cells corresponding to a group of values along the linear scales
 - Example: Query for $\text{Dno} \leq 5$ and $\text{Age} > 40$
 - Refers to the data in the bottom bucket shown in the grid
 - Grid file concept can be applied to any number of search keys
 - For n search keys, the grid array would have n dimensions
 - Grid files perform well in terms of reduction in time for multiple key access
 - Problem is
 - They represent a space overhead in terms of the grid array structure
 - Also, with dynamic files, a frequent reorganization of the file adds to the maintenance cost

Outline

- Introduction
- Types of Single-Level Ordered Indexes
- Multilevel Indexes
- Dynamic Multilevel Indexes Using B-Trees and B+-Trees
- Indexes on Multiple Keys
- **Other Types of Indexes**
- Some General Issues Concerning Indexing

Other Types of Indexes

Outline

- Introduction
- Types of Single-Level Ordered Indexes
- Multilevel Indexes
- Dynamic Multilevel Indexes Using B-Trees and B+-Trees
- Indexes on Multiple Keys
- Other Types of Indexes
- **Some General Issues Concerning Indexing**

Some General Issues Concerning Indexing